BUENAS PRACTICAS DE DESARROLLO DE API

DATOS DE LA VERSIÓN

Documento que sustenta el cambio	Versión	Aprobado por:	Nro.	Fecha de Aprobación	Fecha de Vigencia
	1	Jefe del Dpto. de Desarrollo de Software	001	04/11/2026	04/11/2026
RESPONSABLE DE ACTUALIZACION	Sub Jefe de SISTEMAS				
RESPONSABLE DE REVISION	Jefe de SISTEMAS				

CAPITULO I

1. GENERALIDADES

1.1. OBJETIVO

Establecer los lineamientos para el uso de las Interfaces de Programación de Aplicaciones (API) para la provisión de Servicios en Línea, que son implementados de manera interna o por proveedores externos, a fin de que se contemplen consideraciones adecuadas para el desarrollo, pruebas y despliegue a producción de las API's para los sistemas de La Institución.

1.2. ALCANCE

El Manual de Uso de las Interfaces de Programación de Aplicaciones (API) para la provisión de Servicios en Línea, se aplica a toda implementación y mantenimiento de API's para los Sistemas Informáticos de La Institución, que son implementados por el Departamento de Sistemas y/o por proveedores de desarrollo de software externos, esto implica:

- Análisis de riesgos asociados e implementar las medidas de mitigación.
- La autenticación mutua de los sistemas y la de los usuarios.
- La autorización de las operaciones por parte de los usuarios.
- El cifrado de datos en almacenamiento o transmisión.
- Prácticas de desarrollo seguro de API y revisión de prácticas de codificación segura.
- Análisis de vulnerabilidades y pruebas de penetración.
- La seguridad de la infraestructura tecnológica que lo soporta.
- Los mecanismos de tolerancia ante fallos y de contingencia.

Todo debe estar alineado al desarrollo, pruebas y despliegue a producción de estas API's para su uso en los diversos sistemas informáticos de la INSTITUCIÓN.

CAPITULO II

2. PRINCIPIOS

2.1. Clasificación De Activos

Las Interfaces de Programación De Aplicaciones (Api) van a variar dependiendo del sistema donde se necesite aplicar. Adicional a la parte general que incluye todos los sistemas.

- **Matriz de Clasificación de Datos en Tránsito (Ley N° 29733 y Ley N° 26702):**
    - Nivel 1 (Secreto Bancario / Financiero): Saldos, números de cuenta, códigos CCI, movimientos de cuenta, saldos deudores, historial crediticio.
    - Nivel 2 (Datos Personales Sensibles): Biometría, patrones faciales, huellas dactilares, firma digital, documento nacional de identidad (DNI).
    - Nivel 3 (Confidencial Corporativo): Códigos de agencias internas, identificadores de clientes de base de datos (client_uuid), contratos con comercios o aliados.
    - Nivel 4 (Público): Tarifarios de comisiones, tasas de interés compensatorias/moratorias (TREA/TCEA), red de cajeros y agencias, simuladores públicos de préstamos.
- **Tipología de Red y Perímetros de Despliegue:**
    - APIs de Perímetro Abierto (Públicas / Clientes finales): Desplegadas en DMZ, protegidas por CDN, WAF y Anti-DDoS.
    - APIs de Ecosistema / Finanzas Abiertas (B2B, Fintech, Alianzas): Consumidas vía túneles VPN IPsec dedicados o mTLS obligatorio con certificados emitidos por AC reconocidas.
    - APIs de Tráfico Este-Oeste (Microservicios internos / Core): Dentro de redes privadas (VNet/VPC) aisladas sin acceso directo a internet, gestionadas por mallas de servicios (Service Mesh).

2.2. Consideración de los pilares de la seguridad y Principios de Diseño

La seguridad de información se mantiene sobre los pilares de confidencialidad, integridad y disponibilidad. En relación a ello, el uso de las interfaces contempla también estos pilares para la guía de acciones a realizar.

- **Autenticidad y No Repudio (Exigencia Res. SBS 504):** Garantizar la identidad inequívoca del emisor mediante firmas digitales asimétricas en transacciones de movimiento de fondos.
- **Principio de Mínimo Privilegio (PoLP):** Cuentas de servicio y consumidores restringidos a nivel de endpoint (POST /transferencias vs GET /transferencias/{id}) mediante asignación granular de Scopes OAuth 2.0.
- **Defensa en Profundidad y Confianza Cero (Zero Trust):** Ningún microservicio interno asume que el tráfico dentro de la red corporativa es legítimo; cada llamada requiere validación de token y canal cifrado.

- **Diseño Orientado al Contrato (API-First / Contract-Driven):** El contrato en OpenAPI Specification (OAS 3.1) o AsyncAPI se versiona y aprueba previo al inicio del desarrollo backend.

2.3. Responsables de la ejecución del manual de uso de las interfaces de programación de aplicaciones (API)

Los responsables de la ejecución del manual serán:

- Analista Funcional.
- Analista de desarrollo de software.
- Analista de control de calidad de software.
- Auxiliar de desarrollo de software.
- Auxiliar de control de calidad de software.
- Sub-Jefe de Control de Calidad.
- Empresas de Desarrollo de software.
- Oficial de Seguridad de la Información (CISO) / Ciberseguridad
- Oficial de Cumplimiento / Legal

CAPITULO III

3. ANÁLISIS DE RIESGOS ASOCIADOS A LA IMPLEMENTACIÓN DE API'S.

El análisis de riesgo adecuado de las implementaciones de las API's, se deberá considerar efectuarlo en todas las etapas de los proyectos y/o requerimientos. Debiendo documentar la evidencia de este análisis.

3.1. Riesgos de Planificación.

- No definir de manera especifica todos los problemas dentro del proyecto a desarrollar
- No contar con objetivos claros
- No contar con una propuesta metodológica adecuada
- No contar con un adecuado cronograma de plan de trabajo

3.2. Mitigación de Riesgos de Planificación.

Se tienen las siguientes tareas a ejecutar:

- Tener Análisis de datos administrativos.
- Identificar el nivel de importancia de cada problema.
- Tener disponibilidad inmediata de información por parte de los usuarios expertos.
- Elaborar los objetivos en base a la clasificación por el nivel de importancia del árbol de problemas.
- Diseño Semántico RESTful y Protocolos:
- **Estándar de nombres:** URLS con sustantivos en plural (/v1/creditos, /v1/clientes/{id}/cuentas), uso de minúsculas y guiones medios (kebab-case).
- **Uso correcto de verbos HTTP:**
    - GET: Solo lectura, sin mutación de estado, idempotente y almacenable en caché cuando aplique.
    - POST: Creación de transacciones o recursos dependientes.
    - PUT: Actualización completa e idempotente.
    - PATCH: Actualización parcial controlada.
    - DELETE: Baja lógica obligatoria (en el sector financiero se prohíbe el borrado físico de tablas transaccionales por auditoría).
- **Manejo Estándar de Errores Técnicos (RFC 7807 / RFC 9457 - Problem Details):**

- Toda API debe devolver fallos bajo un esquema estandarizado JSON

```
{
  "type": "https://api.institución.com/errores/saldo-insuficiente",
  "title": "Saldo insuficiente",
  "status": 422,
  "detail": "La cuenta origen no cuenta con fondos suficientes para
debitar el monto solicitado.",
  "instance": "/v1/transferencias/trn-984321",
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}
```

- Prohibición absoluta de devolver códigos 200 OK acompañados de mensajes de error de negocio en el cuerpo.
- **Garantía de Idempotencia en Operaciones Financieras:**
  - Cabecera obligatoria Idempotency-Key (UUIDv4) en todos los endpoints que generen cargos, pagos, desembolsos o transferencias.
  - El backend debe persistir la llave de idempotencia en un almacén en memoria (ej. Redis) con tiempo de vida definido (TTL de 24 a 48 horas) para evitar duplicidad de cobros ante caídas de conexión o reintentos de red.

Existen diversas metodologías como: nada, api first, scrum, etc. En cualquier metodología a usar se debe considerar protocolos binarios en ASN o más pesados como SOAP, técnicamente la API no convierte un servicio en micro-servicio. Si el servicio no tiene un buen nivel de autonomía.

Se deben establecer tiempos ideales para realizar cada tarea señalada en el primer párrafo de la sección 3.2., evitando demoras y favoreciendo que las actividades se ejecuten con la consistencia necesaria.

3.3. Riesgos en el Desarrollo, Pruebas y Despliegue a Producción.

- Configuración incorrecta de API.
- Explotación de mecanismos de autenticación modernos.
- Evolución de los ataques de malware tradicionales para utilizar más API en la nube.
- Posible uso indebido de las API para lanzar ataques a datos empresariales.
- El uso del api para infraestructura definida por software también significa un posible uso indebido
- Fuga de Secretos en Control de Versiones: Prohibición estricta de cadenas de conexión, API keys o certificados en código. Uso mandatorio de pre-commit hooks (TruffleHog, Gitleaks) para escanear repositorios.
- Ataques a la Cadena de Suministro (Supply Chain):
- Uso de registros de artefactos privados con escaneo de vulnerabilidades.
- Degradación por Dependencias en Cascada: Mitigación mediante límites de concurrencia y pools de conexiones dimensionados frente al Core Bancario.

3.4. Mitigación de Vulnerabilidades y Riesgos de Seguridad asociados en la implementación de las API's.

La Fundación Open Web Application Security Project (OWASP) trabaja para mejorar la seguridad del software a través de sus proyectos de software de código abierto liderados por la comunidad, cientos de capítulos en todo el mundo, decenas de miles de miembros y organizando conferencias locales y globales.

El Proyecto de Seguridad API de OWASP se enfoca en estrategias y soluciones para comprender y mitigar las vulnerabilidades únicas y los riesgos de seguridad de las API.

3.5. Vulnerabilidades, Riesgos y Actividades para la mitigación durante la implementación de las API's.

3.5.1. Autorización de nivel de objeto roto

Los objetos de API que no están protegidos con el nivel adecuado de autorización pueden ser vulnerables a fugas de datos y manipulación de datos no autorizada a través de identificadores de acceso de objetos débiles. Por ejemplo, un atacante podría explotar un identificador de objeto entero, que se puede iterar.

Más información sobre esta amenaza ver el Anexo 01 y en el siguiente enlace: API1:2019 Autenticación de nivel de objeto roto

Actividades para la mitigación:

- El mejor lugar para implementar la autorización a nivel de objeto es dentro de la propia API de backend. En el backend, las decisiones de autorización correctas se pueden tomar a nivel de solicitud (u objeto), cuando corresponda, utilizando la lógica aplicable al dominio y la API. Considere escenarios en los que una solicitud determinada puede generar diferentes niveles de detalle en la respuesta, según los permisos y la autorización del solicitante.
- Si una API vulnerable actual no se puede cambiar en el back-end, la gestión de API podría usarse como respaldo.
- Use una política personalizada para implementar la autorización a nivel de objeto, si no está implementada en el backend.
- Implemente una política personalizada para asignar identificadores de la solicitud al backend y del backend al cliente, de modo que los identificadores internos no queden expuestos.
- En estos casos, la política personalizada podría ser una expresión de política con una búsqueda (por ejemplo, un diccionario) o integración con otro servicio a través de la política de envío de solicitudes.
- Para escenarios de GraphQL, aplique la autorización a nivel de objeto a través de la política de solicitud de validación de GraphQL, utilizando el authorize element.

3.5.2. Autenticación de usuario roto

Un buen diseño de interfaz API es engañosamente desafiante. A menudo, particularmente con las API heredadas que han evolucionado con el tiempo, las interfaces de solicitud y respuesta contienen más campos de datos de los que requieren las aplicaciones de consumo.

Un mal actor podría intentar acceder a la API directamente (quizás reproduciendo una solicitud válida) o rastrear el tráfico entre el servidor y la API. El análisis de las acciones de la API y los datos disponibles podrían generar datos confidenciales para el atacante, que la aplicación front-end no revela ni utiliza.

Más información sobre esta amenaza: API2:2019 Autenticación de usuario roto.

Actividades para la mitigación:

- **Autenticación:** API Management admite los siguientes métodos de autenticación:
    - Política de autenticación básica: credenciales de nombre de usuario y contraseña.
    - Clave de suscripción: una clave de suscripción proporciona un nivel de seguridad similar al de la autenticación básica y puede no ser suficiente por sí sola. Si la clave de suscripción se ve comprometida, un atacante puede obtener acceso ilimitado al sistema.
    - Política de certificados de clientes: el uso de certificados de clientes es más seguro que las credenciales básicas o la clave de suscripción, pero no permite la flexibilidad que brindan los protocolos de autorización basados en tokens, como OAuth 2.0.
- **Autorización:** API Management admite una política de validación de JWT para verificar la validez de un token de acceso de OAuth 2.0 JWT entrante en función de la información obtenida del extremo de metadatos del proveedor de identidad de OAuth. Configure la política para verificar los reclamos de token relevantes, la audiencia y el tiempo de vencimiento. Obtenga más información sobre cómo proteger una API mediante la autorización OAuth 2.0 y Azure Active Directory.
- Utilice políticas de restricción de acceso en API Management para aumentar la seguridad. Por ejemplo, la limitación de la tasa de llamadas ralentiza a los malhechores que utilizan ataques de fuerza bruta para comprometer las credenciales.
- Las API deben usar TLS/SSL (seguridad de transporte) para proteger las credenciales o tokens. Las credenciales y los tokens deben enviarse en encabezados de solicitud y no como parámetros de consulta.
- En el portal para desarrolladores de API Management, configure Azure Active Directory o Azure Active Directory B2C como proveedor de

identidad para aumentar la seguridad de la cuenta. El portal para desarrolladores usa CAPTCHA para mitigar los ataques de fuerza bruta.

3.5.3. Exposición excesiva de datos

Un buen diseño de interfaz API es engañosamente desafiante. A menudo, particularmente con las API heredadas que han evolucionado con el tiempo, las interfaces de solicitud y respuesta contienen más campos de datos de los que requieren las aplicaciones de consumo.

Un mal actor podría intentar acceder a la API directamente (quizás reproduciendo una solicitud válida) o rastrear el tráfico entre el servidor y la API. El análisis de las acciones de la API y los datos disponibles podrían generar datos confidenciales para el atacante, que la aplicación front-end no revela ni utiliza.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API3:2019 Exposición excesiva de datos

Actividades para la mitigación:

- El mejor enfoque para mitigar esta vulnerabilidad es garantizar que las interfaces externas definidas en la API de back-end se diseñen cuidadosamente e, idealmente, independientemente de la persistencia de los datos. Deben contener solo los campos requeridos por los consumidores de la API. Las API deben revisarse con frecuencia y los campos heredados deben quedar obsoletos y luego eliminarse.

En Gestión de API, utilice:

  - **Revisiones** para controlar con gracia los cambios no importantes, por ejemplo, la adición de un campo a una interfaz. Puede usar revisiones junto con una implementación de control de versiones en el back-end.
  - **Versiones** para romper cambios, por ejemplo, la eliminación de un campo de una interfaz.

- Si no es posible modificar el diseño de la interfaz de back-end y el exceso de datos es un problema, use las políticas de transformación de API Management para reescribir las cargas útiles de respuesta y enmascarar o filtrar datos. Por ejemplo, elimine las propiedades JSON innecesarias de un cuerpo de respuesta.
- La validación de contenido de respuesta en API Management se puede usar con un esquema XML o JSON para bloquear respuestas con propiedades no documentadas o valores incorrectos. La política también admite el bloqueo de respuestas que excedan un tamaño específico.
- Utilice la política de código de estado de validación para bloquear respuestas con errores no definidos en el esquema de la API.

- Utilice la política de validación de encabezados para bloquear respuestas con encabezados que no estén definidos en el esquema o que no cumplan con su definición en el esquema. Elimine los encabezados no deseados con la política de encabezados establecidos.
- Para escenarios de GraphQL, use la política de solicitud de validación de GraphQL para validar las solicitudes de GraphQL, autorizar el acceso a rutas de consulta específicas y limitar el tamaño de la respuesta.
- Prohibición absoluta del retorno directo de entidades de persistencia (ORM, Data Tables).
- Uso mandatorio de Data Transfer Objects (DTO) de salida con listas blancas explícitas de atributos.

3.5.4. Falta de recursos y limitación de tasas

La falta de limitación de velocidad puede provocar la filtración de datos o ataques DDoS exitosos en los servicios de back-end, lo que provoca una interrupción para todos los consumidores.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API4:2019 Falta de recursos y limitación de velocidad.

Actividades para la mitigación:

- Utilice políticas de límite de tasa (corto plazo) y límite de cuota (largo plazo) para controlar la cantidad permitida de llamadas a la API o el ancho de banda por consumidor.
- Defina definiciones estrictas de objetos de solicitud y sus propiedades en la definición de OpenAPI. Por ejemplo, defina el valor máximo para paginar enteros, maxLength y expresión regular (regex) para cadenas. Aplique esos esquemas con las políticas de validación de contenido y validación de parámetros en API Management.
- Aplique el tamaño máximo de la solicitud con la política de validación de contenido.
- Optimice el rendimiento con el almacenamiento en caché incorporado, lo que reduce el consumo de CPU, memoria y recursos de red para ciertas operaciones.
- Hacer cumplir la autenticación para las llamadas a la API (consulte Autenticación de usuario rota). Revocar el acceso a usuarios abusivos. Por ejemplo, desactive la clave de suscripción, bloquee la dirección IP con la política de restricción de IP de llamadas o rechace las solicitudes para un reclamo de usuario determinado de un token JWT.

- Aplique una política de CORS para controlar los sitios web que pueden cargar los recursos servidos a través de la API. Para evitar configuraciones demasiado permisivas, no use valores comodines (*) en la política de CORS.
- Minimice el tiempo que tarda un servicio back-end en responder. Cuanto más tarde el servicio back-end en responder, más tiempo estará ocupada la conexión en API Management, lo que reduce la cantidad de solicitudes que se pueden atender en un período de tiempo determinado.

    - Definir timeout en la política de solicitud de reenvío.
    - Use la política de solicitud de validación de GraphQL para las API de GraphQL y configure max-depth y configure max-size de los parámetros.
    - Limite la cantidad de conexiones back-end paralelas con la política de límite de simultaneidad.

- Si bien API Management puede proteger los servicios de back-end de los ataques DDoS, puede ser vulnerable a esos ataques. Implemente un servicio de protección contra bots frente a API Management (por ejemplo, Azure Application Gateway, Azure Front Door o Azure DDoS Protection) para protegerse mejor contra los ataques DDoS. Cuando use un WAF con Azure Application Gateway o Azure Front Door, considere usar Microsoft_BotManagerRuleSet_1.0.
- Estrategia de paginación obligatoria por cursor o por rango: limit y offset (con un max_limit forzado en backend, ej. 50 registros por página).
- Límites de carga de solicitud (Request payload limits): Restricción de tamaño máximo del cuerpo HTTP a nivel de Gateway (ej. máximo 2 MB para JSON, 10 MB para documentos en base64).

3.5.5. Autorización de nivel de función rota

Las políticas de control de acceso complejas con diferentes jerarquías, grupos y roles, y una separación poco clara entre las funciones administrativas y regulares conducen a fallas de autorización. Al explotar estos problemas, los atacantes obtienen acceso a los recursos o funciones administrativas de otros usuarios.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API5:2019 Autorización de nivel de función rota

Actividades para la mitigación:

- De forma predeterminada, proteja todos los puntos finales de API en API Management con claves de suscripción.

- Defina una política de validación de JWT y haga cumplir los reclamos de token requeridos. Si ciertas operaciones requieren una aplicación de reclamaciones más estricta, defina validate-jwt políticas adicionales solo para esas operaciones.
- Use una red virtual de Azure o Private Link para ocultar los puntos finales de la API de Internet. Obtenga más información sobre las opciones de red virtual con API Management.
- No defina operaciones de API comodín (es decir, API "catch-all" con * la ruta). Asegúrese de que API Management solo atienda solicitudes para puntos finales definidos explícitamente y que se rechacen las solicitudes para puntos finales no definidos.
- No me publiques API con productos abiertos que no requieran una suscripción.

3.5.6. Asignación masiva

Si una API ofrece más campos de los que requiere el cliente para una acción determinada, un atacante puede inyectar propiedades excesivas para realizar operaciones no autorizadas en los datos. Los atacantes pueden descubrir propiedades no documentadas al inspeccionar el formato de las solicitudes y respuestas u otras API, o al adivinarlas. Esta vulnerabilidad es especialmente aplicable si no utiliza lenguajes de programación fuertemente tipados.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API6:2019 Asignación masiva

Actividades para la mitigación:

- Las interfaces API externas deben desacoplarse de la implementación de datos internos. Evite vincular contratos de API directamente con contratos de datos en servicios de backend. Revise el diseño de la API con frecuencia y desactive y elimine las propiedades heredadas mediante el control de versiones en API Management.
- Defina con precisión contratos XML y JSON en el esquema de la API y utilice políticas de validación de contenido y parámetros para bloquear solicitudes y respuestas con propiedades no documentadas. El bloqueo de solicitudes con propiedades no documentadas mitiga los ataques, mientras que el bloqueo de respuestas con propiedades no documentadas dificulta la ingeniería inversa de posibles vectores de ataque.
- Si la interfaz de back-end no se puede cambiar, use políticas de transformación para reescribir las cargas útiles de solicitud y respuesta y desacoplar los contratos de API de los contratos de back-end. Por ejemplo, enmascare o filtre datos o elimine propiedades JSON innecesarias.

3.5.7. Configuración incorrecta de seguridad

Los atacantes pueden intentar explotar las vulnerabilidades de configuración incorrecta de la seguridad, como:

- Falta refuerzo de seguridad
- Funciones habilitadas innecesarias
- Conexiones de red innecesariamente abiertas a Internet
- Uso de protocolos o cifrados débiles.
- Otras configuraciones o puntos finales que pueden permitir el acceso no autorizado al sistema.
- Cabeceras de seguridad requeridas en cada respuesta HTTP:
    - **Strict-Transport-Security:** max-age=31536000; includeSubDomains; preload
    - **X-Content-Type-Options:** nosniff
    - **X-Frame-Options:** DENY
    - **Content-Security-Policy:** default-src 'none'
    - **Cache-Control:** no-store, no-cache, must-revalidate (en todos los endpoints transaccionales y de consulta de saldos).

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API7:2019 Error de configuración de seguridad.

Actividades para la mitigación:

- Configure correctamente la puerta de enlace TLS. No utilice protocolos vulnerables (por ejemplo, TLS 1.0, 1.1) o cifrados.
- Configure las API para aceptar solo tráfico cifrado, por ejemplo, a través de los protocolos HTTPS o WSS.
- Considere implementar API Management detrás de un extremo privado o conectado a una red virtual implementada en modo interno. En las redes internas, el acceso se puede controlar desde dentro de la red privada (a través de un firewall o grupos de seguridad de la red) y desde Internet (a través de un proxy inverso).
- Use las políticas de Azure API Management:
    - Herede siempre las políticas principales a través de la <base>etiqueta.
    - Cuando utilice OAuth 2.0, configure y pruebe la política de validación de JWT para verificar la existencia y validez del token de JWT antes de que llegue al back-end. Verifique automáticamente el tiempo de vencimiento del token, la firma del token y el emisor. Haga cumplir las reclamaciones, las audiencias, la caducidad del token y la firma del token a través de la configuración de políticas.
    - Configure la política de CORS y no use comodines * para ninguna opción de configuración. En su lugar, enumere explícitamente los valores permitidos.

    - Establezca políticas de validación en prevent entornos de producción para validar esquemas JSON y XML, encabezados, parámetros de consulta y códigos de estado, y para hacer cumplir el tamaño máximo de solicitud o respuesta.
    - Si API Management está fuera de los límites de la red, la validación de la IP del cliente aún es posible mediante la política de restricción de IP de la persona que llama. Asegúrese de que utilice una lista de permitidos, no una lista de bloqueados.
    - Si se utilizan certificados de cliente entre la persona que llama y API Management, utilice la política de validación de certificados de cliente. Asegúrese de que los atributos validate - revocation, validate - trust, validate - not - beforey validate - not - afterestén configurados en true.

    - Los certificados de cliente (TLS mutuo) también se pueden aplicar entre API Management y el back-end. El back-end debería:

        - Tener credenciales de autorización configuradas.
        - Validar la cadena de certificados cuando corresponda.
        - Valide el nombre del certificado cuando corresponda.

- Para escenarios de GraphQL, use la política de solicitud de validación de GraphQL. Asegúrese de que el authorization elemento y max - size los max-depth atributos estén establecidos.
- No almacene secretos en archivos de políticas o en el control de código fuente. Utilice siempre valores con nombre de API Management u obtenga los secretos en tiempo de ejecución mediante expresiones de políticas personalizadas.
- Los valores con nombre deben integrarse con Key Vault o cifrarse dentro de API Management marcándolos como "secretos". Nunca almacene secretos en valores con nombre de texto sin formato.
- Publique API a través de productos, que requieren suscripciones. No utilice productos abiertos que no requieran una suscripción.
- Utilice la integración de Key Vault para administrar todos los certificados: esto centraliza la administración de certificados y puede ayudar a facilitar las tareas de administración de operaciones, como la renovación o revocación de certificados.
- Al usar la puerta de enlace auto hospedada, asegúrese de que haya un proceso para actualizar la imagen a la última versión periódicamente.

- Representar los servicios de back-end como entidades de back-end. Configure las credenciales de autorización, la validación de la cadena de certificados y la validación del nombre del certificado cuando corresponda.
- Al utilizar el portal para desarrolladores:
    - Si elige hospedar el portal para desarrolladores, asegúrese de que exista un proceso para actualizar periódicamente el portal hospedado a la última versión. Las actualizaciones de la versión administrada predeterminada son automáticas.
    - Use Azure Active Directory (Azure AD) o Azure Active Directory B2C para el registro e inicio de sesión de usuarios. Deshabilite la autenticación predeterminada de nombre de usuario y contraseña, que es menos segura.
    - Asigne grupos de usuarios a los productos, para controlar la visibilidad de las API en el portal.
- Use Azure Policy para hacer cumplir la configuración de nivel de recursos de API Management y los permisos de control de acceso basado en roles (RBAC) para controlar el acceso a los recursos. Otorgue los privilegios mínimos requeridos a cada usuario.
- Utilice un proceso de DevOps y un enfoque de infraestructura como código fuera de un entorno de desarrollo para garantizar la coherencia del contenido de API Management y los cambios de configuración y para minimizar los errores humanos.
- No utilice ninguna característica obsoleta.

3.5.8. Inyección

Cualquier punto final que acepte datos de usuario es potencialmente vulnerable a un exploit de inyección. Los ejemplos incluyen, pero no se limitan a:

- Inyección de comandos, donde un mal actor intenta alterar la solicitud de API para ejecutar comandos en el sistema operativo que aloja la API.
- Inyección SQL, donde un mal actor intenta alterar la solicitud de API para ejecutar comandos y consultas en la base de datos de la que depende una API.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API8:2019 Inyección.

Actividades para la mitigación:

- Las políticas modernas de firewall de aplicaciones web (WAF) cubren muchas vulnerabilidades de inyección comunes. Si bien API Management no tiene un componente WAF integrado, se recomienda enfáticamente implementar un WAF en sentido ascendente (delante) de

la instancia de API Management. Por ejemplo, use Azure Application Gateway o Azure Front Door.

Importante

Asegúrese de que un mal actor no pueda eludir la puerta de enlace que aloja el WAF y conectarse directamente a la puerta de enlace de API Management o a la propia API de back-end. Las posibles mitigaciones incluyen: ACL de red, uso de la política de administración de API para restringir el tráfico entrante por IP de cliente, eliminación del acceso público donde no sea necesario y autenticación de certificado de cliente (también conocido como TLS mutuo o mTLS).

- Use políticas de validación de esquemas y parámetros, cuando corresponda, para restringir y validar aún más la solicitud antes de que llegue al servicio de la API de back-end.
- El esquema proporcionado con la definición de la API debe tener una restricción de patrón de expresión regular aplicada a los campos vulnerables. Cada expresión regular debe probarse para asegurarse de que restringe el campo lo suficiente como para mitigar los intentos de inyección comunes.

3.5.9. Gestión inadecuada de activos

Las vulnerabilidades relacionadas con la gestión inadecuada de activos incluyen:

- Falta de documentación API adecuada o información de propiedad.
- Cantidades excesivas de versiones anteriores de API, a las que pueden faltar soluciones de seguridad.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API9:2019 Gestión inadecuada de activos

Actividades para la mitigación:

- Utilice una especificación Open API bien definida como fuente para importar API REST. La especificación permite la encapsulación de la definición de la API, incluidos los metadatos autodocumentados.
- Utilice interfaces API con rutas precisas, esquemas de datos, encabezados, parámetros de consulta y códigos de estado. Evite las operaciones con comodines. Proporcione descripciones para cada API y operación e incluya información de contacto y licencia.
- Evite puntos finales que no contribuyan directamente al objetivo empresarial. Aumentan innecesariamente el área de superficie de ataque y dificultan la evolución de la API.

- Use revisiones y versiones en API Management para gobernar y controlar los puntos finales de API. Tenga una sólida estrategia de control de versiones de back-end y comprométase con un número máximo de versiones de API admitidas (por ejemplo, 2 o 3 versiones anteriores). Planee dejar de usar rápidamente y, en última instancia, eliminar las versiones de API más antiguas, a menudo menos seguras.
- Utilice una instancia de API Management por entorno (como desarrollo, prueba y producción). Asegúrese de que cada instancia de API Management se conecte a sus dependencias en el mismo entorno. Por ejemplo, en el entorno de prueba, el recurso de API Management de prueba debe conectarse a un recurso de prueba de Azure Key Vault y las versiones de prueba de los servicios de back-end. Utilice la automatización de DevOps y las prácticas de infraestructura como código para ayudar a mantener la coherencia y la precisión entre los entornos y reducir los errores humanos.
- Use etiquetas para organizar las API y los productos y agrúpelos para su publicación.
- Publique API para su consumo a través del portal para desarrolladores integrado. Asegúrese de que la documentación de la API esté actualizada.
- Descubra API no documentadas o no administradas y expóngalas a través de API Management para un mejor control.

3.5.10. Registro y monitoreo insuficientes

El registro y la supervisión insuficientes, junto con la integración faltante o ineficaz con la respuesta a incidentes, permite a los atacantes atacar aún más los sistemas, mantener la persistencia, pasar a más sistemas para manipularlos y extraer o destruir datos. La mayoría de los estudios de brechas demuestran que el tiempo para detectar una brecha es de más de 200 días, generalmente detectado por partes externas en lugar de procesos internos o monitoreo.

Más información sobre esta amenaza en el Anexo 01 y en el siguiente enlace: API10: 2019 Registro y monitoreo insuficientes.

Actividades para la mitigación:

- Comprenda las opciones de observabilidad en Azure API Management y las mejores prácticas para monitorear en Azure.
- Supervise el tráfico de la API con Azure Monitor.
- Inicie sesión en Application Insights para fines de depuración. Correlacione transacciones en Application Insights entre API Management y la API de back-end para rastrearlas de un extremo a otro.
- Si es necesario, reenvíe eventos personalizados a Event Hubs.

- Establezca alertas en Azure Monitor y Application Insights, por ejemplo, para la métrica de capacidad o para solicitudes excesivas o transferencia de ancho de banda.
- Utilice la política de emisión de métricas para métricas personalizadas.
- Use el registro de actividad de Azure para realizar un seguimiento de la actividad en el servicio.
- Use eventos personalizados en Azure Application Insights y Azure Monitor según sea necesario.
- Configure OpenTelemetry para puertas de enlace auto hospedadas en Kubernetes.
- **Reglas de Enmascaramiento y Sanitización (Cumplimiento SBS y PCI-DSS):**
    - **Dato prohibido en logs:** Contraseñas, PIN, CVV/CVC, tokens completos de autorización (JWT), claves privadas, respuestas de biometría.
    - **Dato enmascarado:** Tarjetas de débito/crédito (mostrar únicamente los primeros 6 y últimos 4 dígitos: 411111******1111), números de cuenta/CCI (mostrar solo últimos 4 caracteres).

3.5.11. Falsificación de Solicitudes del Lado del Servidor (SSRF)

En servicios que reciban webhooks o URLs externas. Implementar resolución DNS segura, listas blancas estrictas y bloqueo de rangos privados (RFC 1918) y endpoints de metadatos de nube (169.254.169.254).

3.5.12. Consumo Inseguro de APIs Externas

Al interactuar con el buró crediticio, pasarelas de pago o entidades de consulta (ej. RENIEC). Validar estrictamente la respuesta antes del procesamiento interno y no reenviar los datos crudos hacia los clientes.

3.5.13. Falla de Lógica de Negocio y Reglas Transaccionales

Mitigación mediante máquinas de estados para evitar transacciones fuera de secuencia (ej. confirmar un crédito sin haber pasado la etapa de validación de identidad).

CAPITULO IV

4. LINEAMIENTOS PARA LA REALIZACIÓN DE PRUEBAS DE SEGURIDAD INFORMÁTICA

4.1. Para exposición de APIs (Proveedor de la API):

4.1.1. Validación de Entradas

Asegurar que la API valide y filtre adecuadamente todas las entradas, incluyendo parámetro de solicitud y encabezados, para prevenir ataques de inyección. Como SQL Injection.

4.1.2. Autenticación y Autorizaciones Sólidas

- Implementar una autenticación robusta y controles de autorización para garantizar que solo los usuarios autorizados accedan a recursos específicos. Verificar la implementación de OAuth, JWT u otros estándares de autenticación segura.
- Uso de OAuth 2.0 (RFC 6749) y OpenID Connect (OIDC) con perfiles de alta seguridad (FAPI - Financial-grade API cuando se expongan servicios abiertos).
- Políticas para Tokens JWT:
    - Algoritmos asimétricos permitidos exclusivamente: RS256, ES256 o PS256. Prohibición estricta del algoritmo none y de algoritmos simétricos basados en secretos compartidos (HS256) en entornos distribuidos.
    - Tiempo de vida de Access Token acotado (máximo 15 minutos). Uso de Refresh Tokens con rotación obligatoria de un solo uso.
    - Validación mandatoria de claims: iss (emisor), aud (audiencia), exp (expiración) y nbf (no antes de).
- mTLS (Mutual TLS): Obligatorio para todas las conexiones entre sistemas Core, redes de cajeros y consumo de socios B2B.

4.1.3. Control de Sesiones

Implementar controles de sesión seguros y verificar que las sesiones se me manejen de manera adecuada para evitar problemas como secuestro de sesión.

4.1.4. Protección contra Ataques Comunes

Asegurar que la API esté protegida contra ataques comunes, como Cross-Site Scripting (XSS) y Cross-Site Forgery (CSRF)

4.1.5. Protección de Datos Sensibles

- Utilizar cifrado adecuado (por ejemplo, TLS) para proteger la confidencialidad de los datos transmitidos y almacena datos sensibles de manera segura.

- Protocolo mínimo obligatorio: TLS 1.2 (con suites cipher suites en modo GCM/DHE) y habilitación preferente de TLS 1.3. Deshabilitar soporte para TLS 1.0, TLS 1.1 y algoritmos CBC, RC4 y 3DES.
- Cifrado en reposo para tablas de base de datos (TDE) y particiones que me almacenen información clasificada de Nivel 1 y 2.

4.1.6. Registro y Auditoría

Implementar registros de seguridad y auditorías para rastrear actividades y detectar posibles amenazas o violaciones de seguridad.

4.2. Para consumo de APIs (Consumidor de la API):

4.2.1. Validación de Datos Recibidos

Validar y filtrar adecuadamente los datos recibidos de la API externa para evitar vulnerabilidades de inyección y otros ataques

4.2.2. Seguridad de Claves API

Asegurar que las claves de API se almacenen y transmitan de manera segura (de acuerdo a la documentación que proporciona el proveedor). No se deben exponer en repositorios públicos o en el código fuente.

4.2.3. Control de Errores

- Validar y gestionar adecuadamente los errores y excepciones que provienen de la API externa para evitar revelaciones de información sensible.
- Patrón Circuit Breaker: Abrir el circuito cuando la tasa de fallo de una API externa supere un umbral (ej. 50% de errores en una ventana de 10 segundos) para no me degradar los hilos de ejecución de la aplicación interna.

4.2.4. Validación de Respuestas

- Validar las respuestas de la API externa para garantizar que no contengan datos maliciosos o no confiables.
- Estrategia de Timeouts Mandatoria: Prohibir clientes HTTP sin timeout explícito:
    - Connect Timeout: Máximo 2 a 3 segundos.
    - Read/Response Timeout: Máximo 5 a 8 segundos en operaciones síncronas.
- Reintentos Inteligentes con Retroceso Exponencial (Exponential Backoff con Jitter): Evitar sobrecargar los servicios en recuperación enviando avalanchas de solicitudes en el mismo milisegundo.

4.2.5. Protección contra Ataques Comunes

Asegurar que las solicitudes a la API externa estén protegidas contra ataques comunes como CSRF y ataques de inyección.

4.2.6. Seguimiento y Monitorización

Implementar un seguimiento y monitorización constante de las solicitudes y respuestas de la API externa para detectar anomalías o posibles amenazas

4.2.7. Actualización y Parches

Mantener actualizadas todas las bibliotecas y dependencias utilizadas para consumir la API externa, ya que las vulnerabilidades pueden surgir en bibliotecas de terceros.

CAPITULO V

5. BIBLIOGRAFÍA

- Application Security Verification Standard – OWASP.
- Plan de Seguridad para plataformas empleando normas ISO-27001 y considerando el OWASP top 10.
- Dominio principal sobre API: https://github.com/OWASP/API-security.
- Resolución de la Superintendencia de Banca y Seguros (SBS) N° 504.
- **Superintendencia de Banca, Seguros y AFP (SBS):**
    - Resolución SBS N° 504-2021: Reglamento para la Gestión de la Seguridad de la Información y la Ciberseguridad.
    - Resolución SBS N° 01747-2026: Reglamento para la prestación del modelo Banking as a Service (BaaS).
    - Resolución SBS N° 02519-2026: Estándares de Infraestructura Digital, Seguridad y Calidad para el Sistema de Finanzas Abiertas.
- **Normativa Nacional de Protección de Datos:**
    - Ley N° 29733: Ley de Protección de Datos Personales del Perú y su Reglamento (Decreto Supremo N° 003-2013-JUS).
- **Estándares Internacionales de Industria:**
    - OWASP API Security Top 10 (Edición 2023).
    - NIST SP 800-204: Security Strategies for Microservices-based Application Systems.
    - ISO/IEC 27001:2022: Controles de seguridad A.8.25 (Desarrollo seguro) y A.8.26 (Requisitos de seguridad de las aplicaciones).
    - RFC 7807 / RFC 9457 (IETF): Problem Details for HTTP APIs.
    - OpenAPI Specification v3.1.